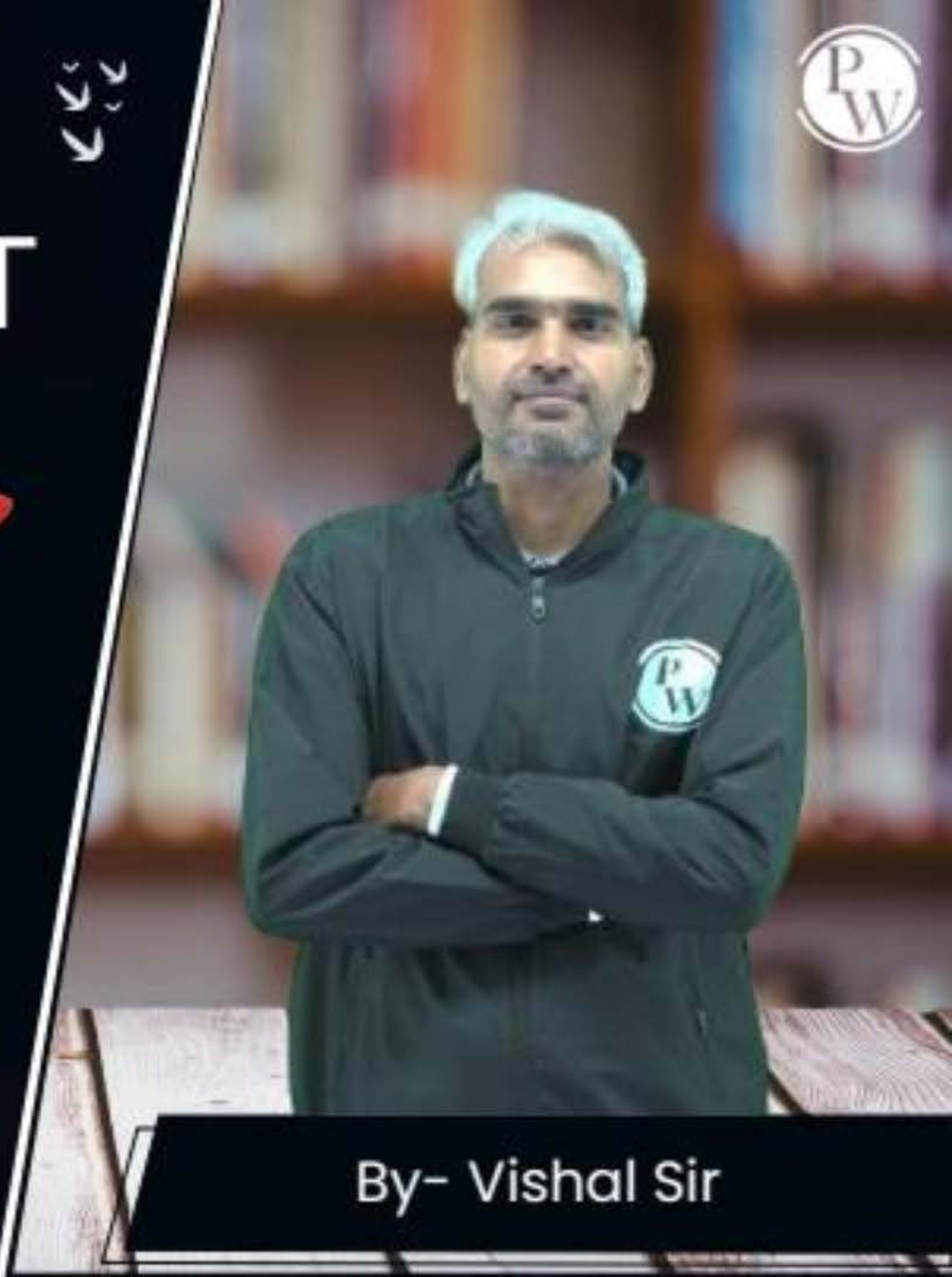
Computer Science & IT



Database Management System

File organization And Indexing

Lecture No. 04



By- Vishal Sir

Recap of Previous Lecture



- Topic: Index file
- Topic: IO cost with index file
- Topic: Categories of index
- Topic: Dense index
- Topic: Sparse index

$\lceil \log_2 M \rceil + 1$

Slide

Topics to be Covered



- Topic: Types of index
- Topic: Primary index
- Topic: Clustering index
- Topic: Secondary index
- Topic: Multi-level index

Slide



Topic : Index file

* Index files are used to reduce the IO cost.

Each entry in the index file contains only two fields

* ① Search key attribute value
* ② Pointer pointing to the record / disk block corresponding to the search key value.

* ① Index file entry = < Search key value , Pointer information >
* ② Index file entry size = (Search key attribute size + Pointer size)
* ③ Database Record Size = (Summation of the sizes required to store each attribute of the database file)
④ Disk block size is same for database file and index file

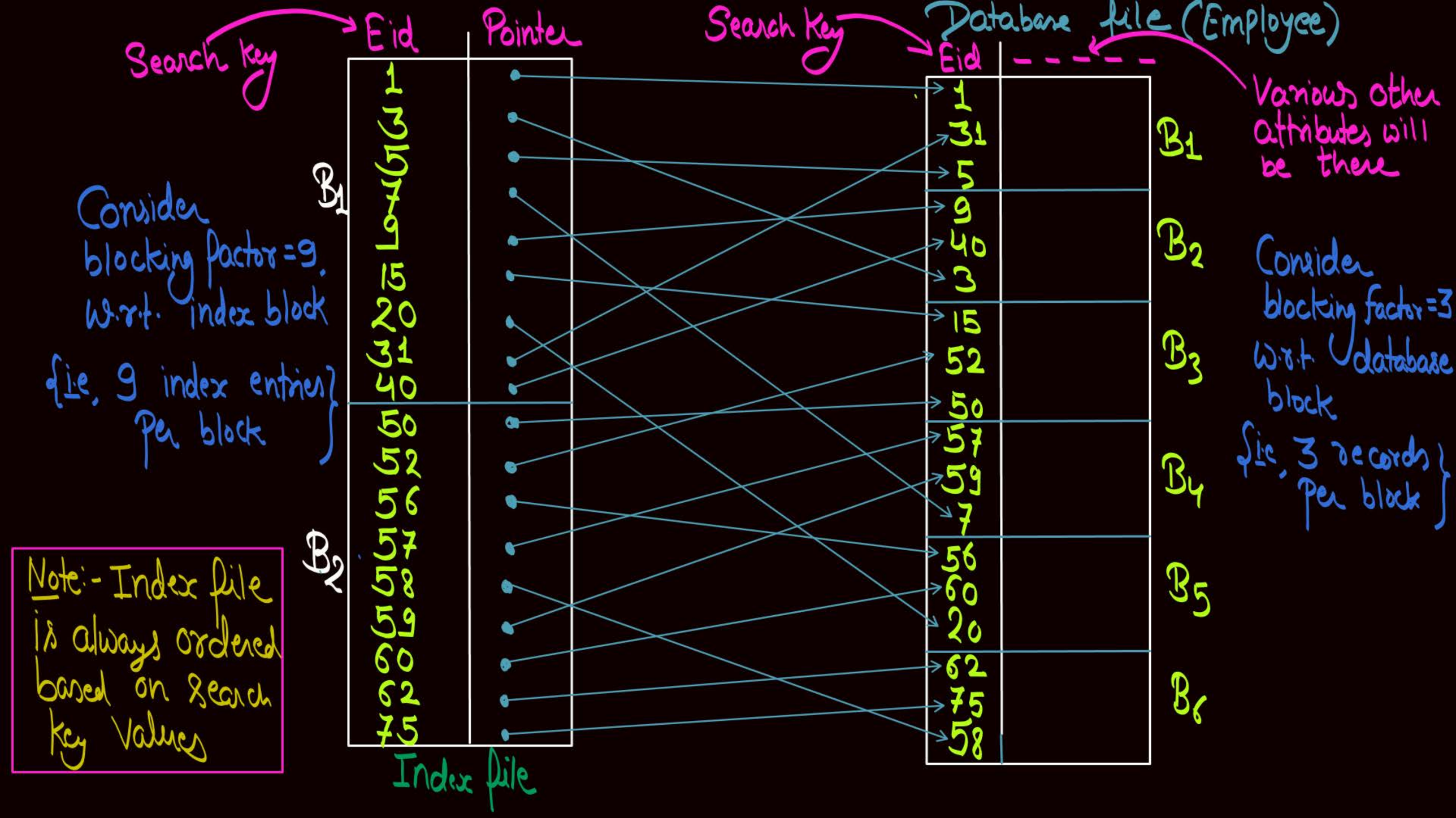
⑤ Blocking factor (w.r.t. database block) = $\frac{\text{Disk block size}}{\text{Record size}}$

⑥ Blocking factor (w.r.t. index block) = $\frac{\text{Disk Block Size}}{\text{Index file entry size}}$ = No. of index entries in each block of disk

⑦ In general, Index file entry size < Database file record size

⑧ By ④, ⑤, ⑥ & ⑦

Blocking factor of index block > Blocking factor w.r.t. database block


Search key
Eid
Pointer
Search key
Database file (Employee)
Eid
Various other attributes will be there
B1
B2
B3
B4
B5
B6
Consider blocking factor = 9. w.r.t. index block
{i.e. 9 index entries} per block
Consider blocking factor = 3 w.r.t. database block
{i.e. 3 records} per block
Note:- Index file is always ordered based on search key values
Index file

⑨ Number of disk blocks required to store database file

$$= \left\lceil \frac{\text{Total number of records in database file}}{\text{Number of records per block}} \right\rceil$$

$$= \left\lceil \frac{\text{Total number of records in database file}}{\text{Blocking factor w.r.t. database block}} \right\rceil$$

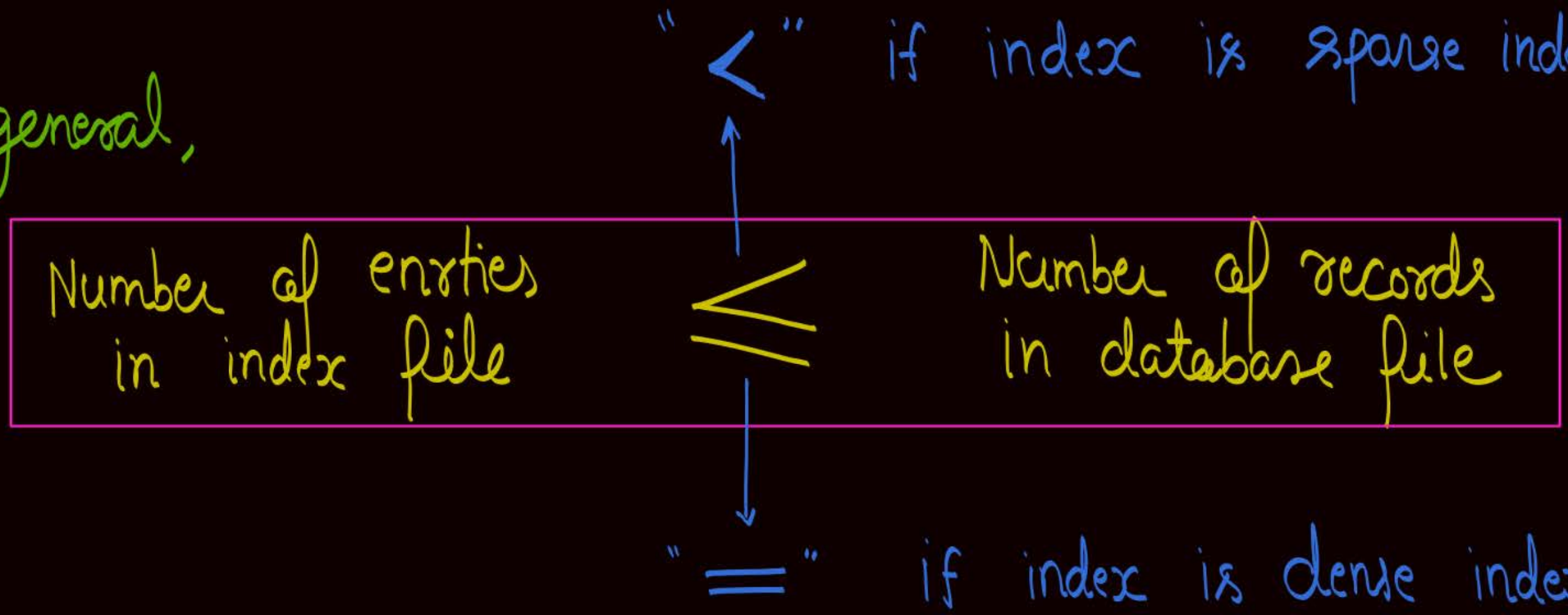
---

⑩ Number of disk blocks required to store index file

$$= \left\lceil \frac{\text{Total number of entries in index file.}}{\text{Number of entries per block}} \right\rceil$$

$$= \left\lceil \frac{\text{Total number of entries in index file}}{\text{Blocking factor w.r.t index block.}} \right\rceil$$

(11) In general,



(12) Index file will always be ordered based on search key attribute values.

→ ∴ We can always perform "binary search" in index file



Topic : IO cost with indexing

- Let 'M' is the number of disk blocks required to store index file, and N is the number of disk blocks required to store the database file. then in general, $M << N$

$$\text{Worst case IO Cost with index file} = \lceil \log_2 M \rceil + 1$$

(to search for an entry in the index file w.r.t. given search key)

To access the block of database file, that contains the record corresponding to the search key value



Topic : Categories of Index

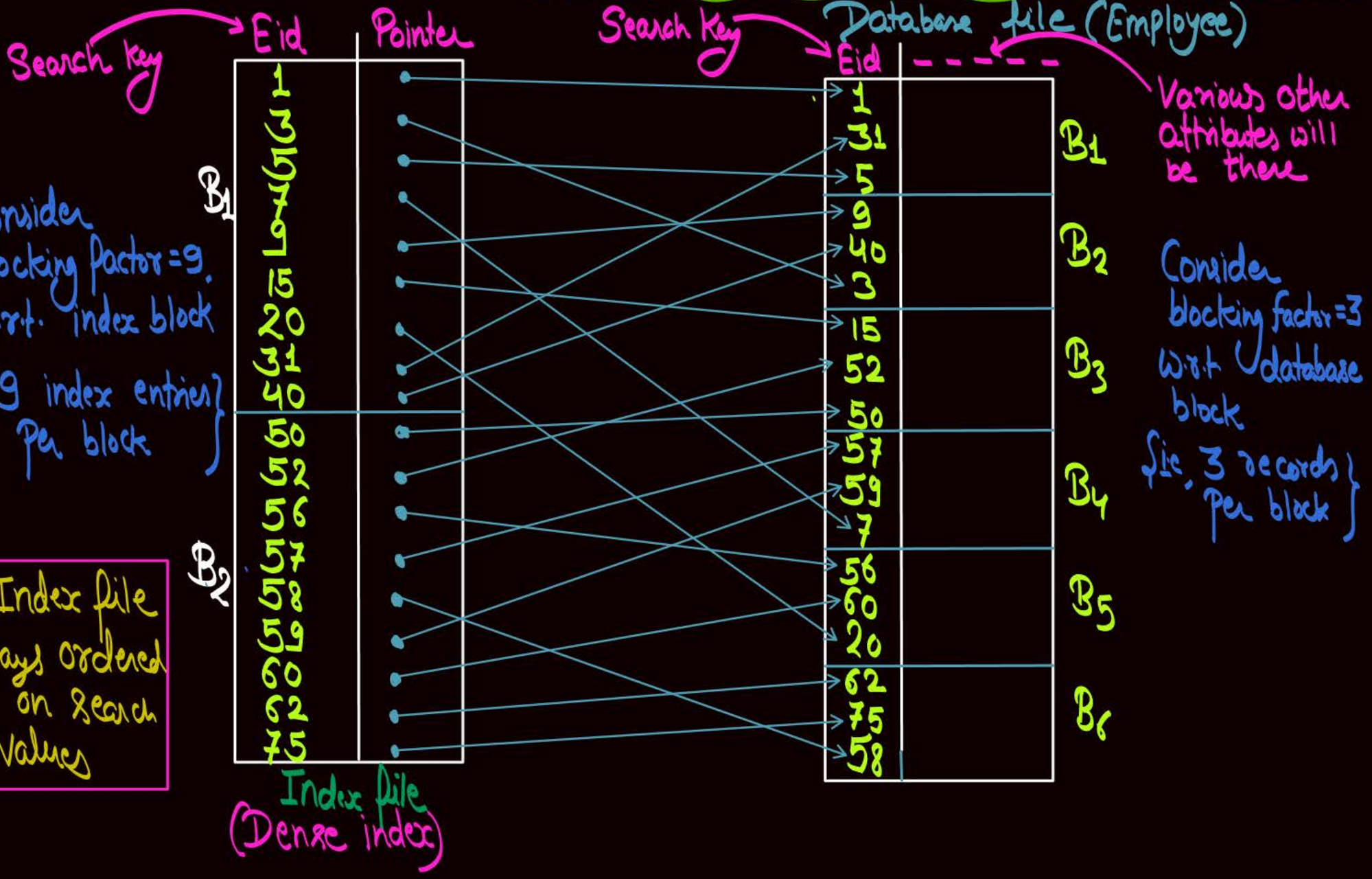
There are two categories of the indexes

1. **Dense index** :- If we maintain an entry in the index file for each record of the database file, then it is called dense index.

i.e; Number of entries in index file w.r.t. dense index = No. of records in the database file

Example of dense index :-

In general, dense index is created when file is unordered based on search key



Note :- Dense index can be created for ordered file as well, but generall it is created for unordered file



Topic : Categories of Index

There are two categories of the indexes

② Sparse index :- For a collection of records in the database file we maintain only one entry in index file, then it is sparse index

i.e; Number of entries in index file w.r.t. Sparse index < No. of records in the database file

* If index is "sparse index" and nothing is specified in the question about the number of entries in the sparse index file, then in gener "1" index file entry is used for each block of database file

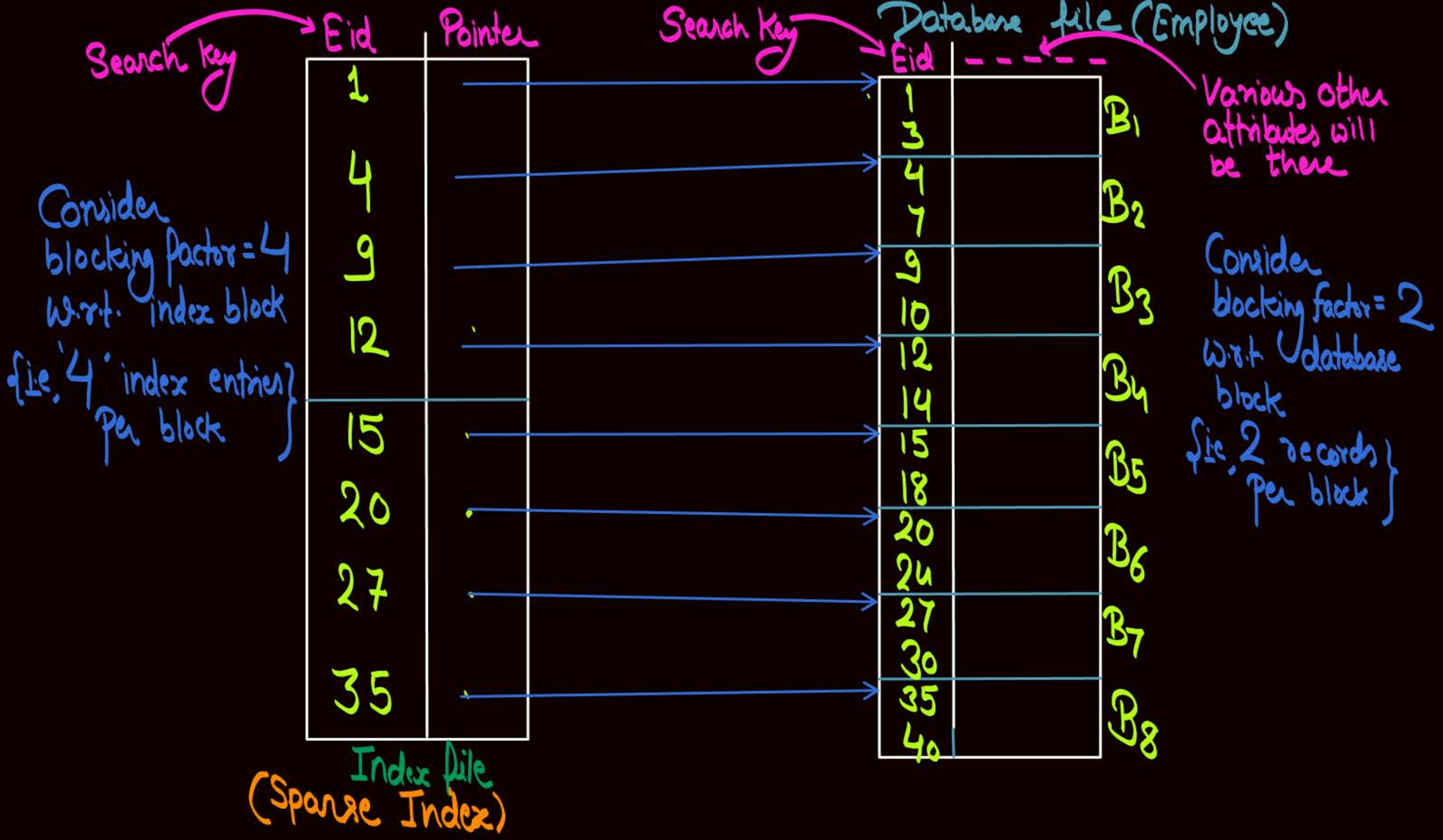
i.e; Number of entries in Sparse index = No. of disk blocks required to store database file

* Example of Sparse index file:-

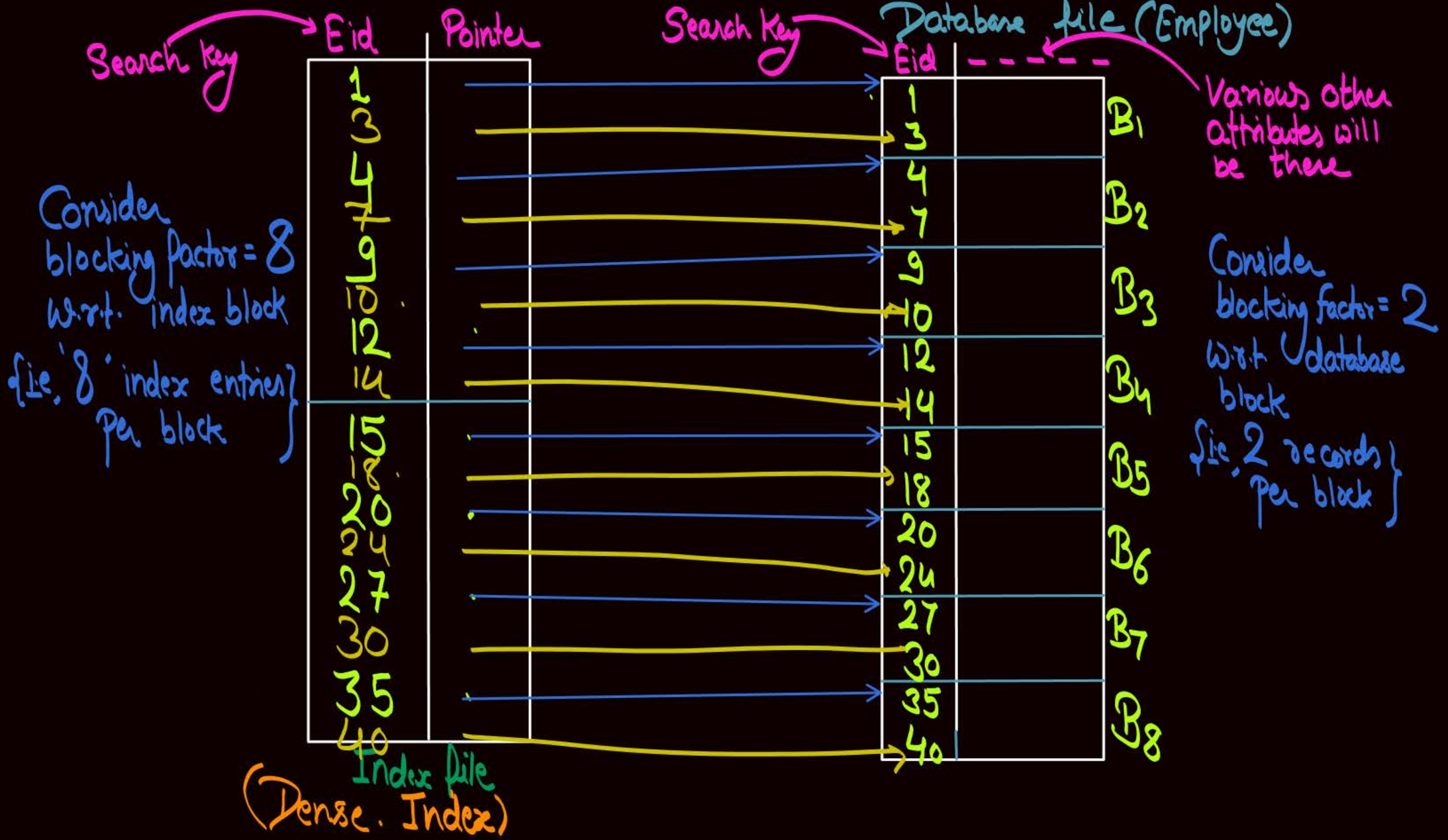


* Example of Sparse index file:-

Sparse index can be created only for ordered file (Not possible for unordered file)



* Example of Dense index file w.r.t. ordered file :-



Note :-

① If file is ordered based on search key attribute values, then both Sparse as well as dense index are possible

② If file is Unordered based on search key attribute values, then only dense index is Possible on that search key

{ Sparse index is never possible for an unordered file }

Q:- Consider the following specifications.

Disk block size = 1000 Bytes

Record size = 100 Bytes

Search key attribute size: 12 Bytes

Record/Block pointer size = 8 Bytes

let total no. of records in the database file = 10,000

(1) How many disk blocks are required to store the dense index file on the above database file

(2) How many disk blocks are required to store the sparse index file on the above database file

① No. of disk blocks required to stored dense index file

$$= \frac{\text{Total no. of entries in dense index file}}{\text{Blocking factor w.r.t. dense index file}}$$

$$= \frac{\text{Total no. of records in database file}}{\left(\frac{\text{Block size}}{\text{index file entry size w.r.t. dense index}}\right)} \quad \left(\text{Search key size} + \text{Record Pointer size}\right)$$

$$= \frac{10,000}{\left(\frac{1000}{(12+8)}\right)} = \frac{10,000}{(50)} = \boxed{200}$$

Ans

② Number of disk blocks required to store sparse index file = Total no. of Entries in Sparse index / Blocking factor w.r.t. Sparse index

= No. of disk blocks required to store database file / Blocking factor w.r.t. Sparse index

= # Records in database file / Blocking factor w.r.t database file

= 10,000 / (Block Size / Record Size) = 10,000 / (1000 / 100)

= 1000

= 1000 / (Block Size / (Search key size + Block ptr size))

= 1000 / (1000 / (12 + 8))

= 1000 / 50 = 20 Ans

Note: ① Generally, we use record pointers in dense index

∴ Index file entry size w.r.t dense index = (Search key size + Record pointer size)

② Generally, we use record pointers in sparse index

∴ Index file entry size w.r.t sparse index = (Search key size + Block pointer size)

Note: If only one pointer is given then use same for both dense index as well as sparse index



Topic : Types of Index

- Single level index
  - Primary index
  - Clustering index
  - Secondary index
- Multi-level index
- Dynamic Balanced Search tree (B tree & B+ tree)



Topic : Primary Index

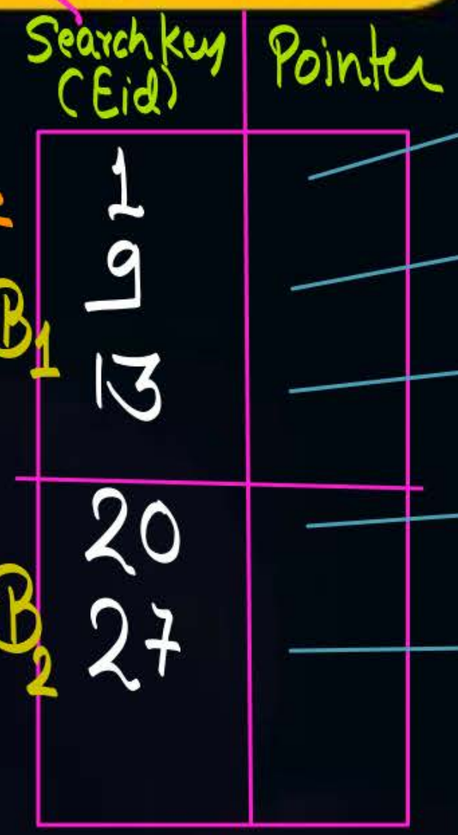
- If ① Search key attribute is a key {i.e.: Unique values}
  and ② File is ordered based on that search key attribute,
  then Index file created on such search key
  is called primary index.



Topic : Primary Index

Eid is a key attribute in database file (i.e: unique values of Eid) & Records of EMP database file are ordered based on Eid

∴ Index is Primary index



Index file.
Primary index
{It can be dense or sparse}

Search key → Eid ← Unique

E.M.P.

Eid	E.M.P.	
1		B_1
7		
9		B_2
10		
13		B_3
16		
20		B_4
24		
27		B_5
30		

{In general, Primary index is Sparse index}



Topic : Clustering Index

* If ① Search key attribute is non-key attribute {i.e. not unique values}
- And ② File is ordered on search key attribute,

then index file created on such search key is called a Clustering index



Topic : Clustering Index

Dept-id values are not unique in database file

&

Records of EMP database file are ordered based on Dept-id

∴ Index will be Clustering index

Search key

not unique

EMP

Dept_id	Eid	- - - -
1	7	
2	13	
2	1	
2	5	
4	4	
5	9	
5	15	
7	6	
7	20	
7	12	

B_1 B_2 B_3 B_4 B_5

Block Anchor

Search key Dept-id | Pointer

Search key Dept-id	Pointer
1	
2	
4	
5	
7	

Index file (Clustering Index)



Topic : Clustering Index

Dept-id values are not unique in database file & Records of EMP database file are ordered based on Dept-id

∴ Index will be Clustering index

Search key

not unique

EMP

Dept_id	Eid	- - - -
1	7	
2	13	
2	1	
2	5	
4	4	
5	9	
5	15	
7	6	
7	20	
7	12	

B1 B2 B3 B4 B5

Block Anchor

Search key Dept-id	Pointer
1	
2	
4	
5	
7	

It is the add? of the first record w.r.t - dept-id = 2

If record is the last record of the database block, then we don't know whether it is the last record corresponding to that search key value or not

∴ We will use the block anchor to access the next block of the database file to check if there are more records corresponding to that search key

Note :→ ① Primary index can be sparse index or dense index {generally, Primary index is sparse}

② Clustering index is always a Sparse index.

③ At most one primary index is possible on a database file {because file must be ordered based on search key}

④ At most one Clustering index is possible on a database file. {because file must be ordered based on search key}

Note - ⑤ On a database file we can either have a primary index or we can have a Clustering index but not both.

Because at a time file can be ordered w.r.t. at most one attribute



Topic : Secondary Index

Search key values may or may not be unique in database file

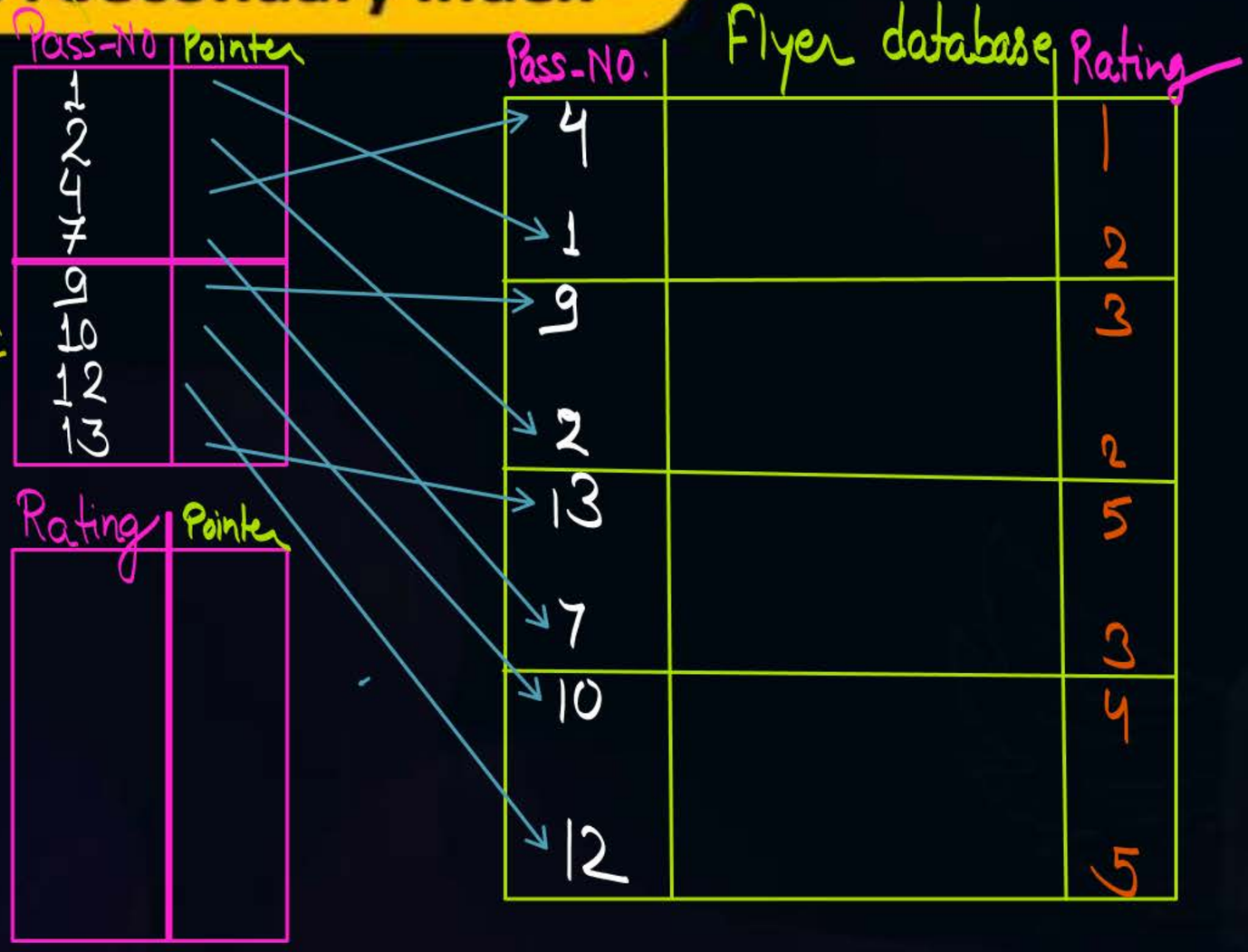
- If file is unordered based on search key;
- then index file created on such search key is called secondary index



Topic : Secondary Index

File is unordered based on Pass-no.

∴ Secondary index



Topic : Secondary Index

File is unordered based on Pass-no.
∴ Secondary index

Database file is unordered based on "Rating"
∴ Secondary index





Note:- When file is unordered based on search key. then our first objective will be to provide an order using index file so that binary search can be applied

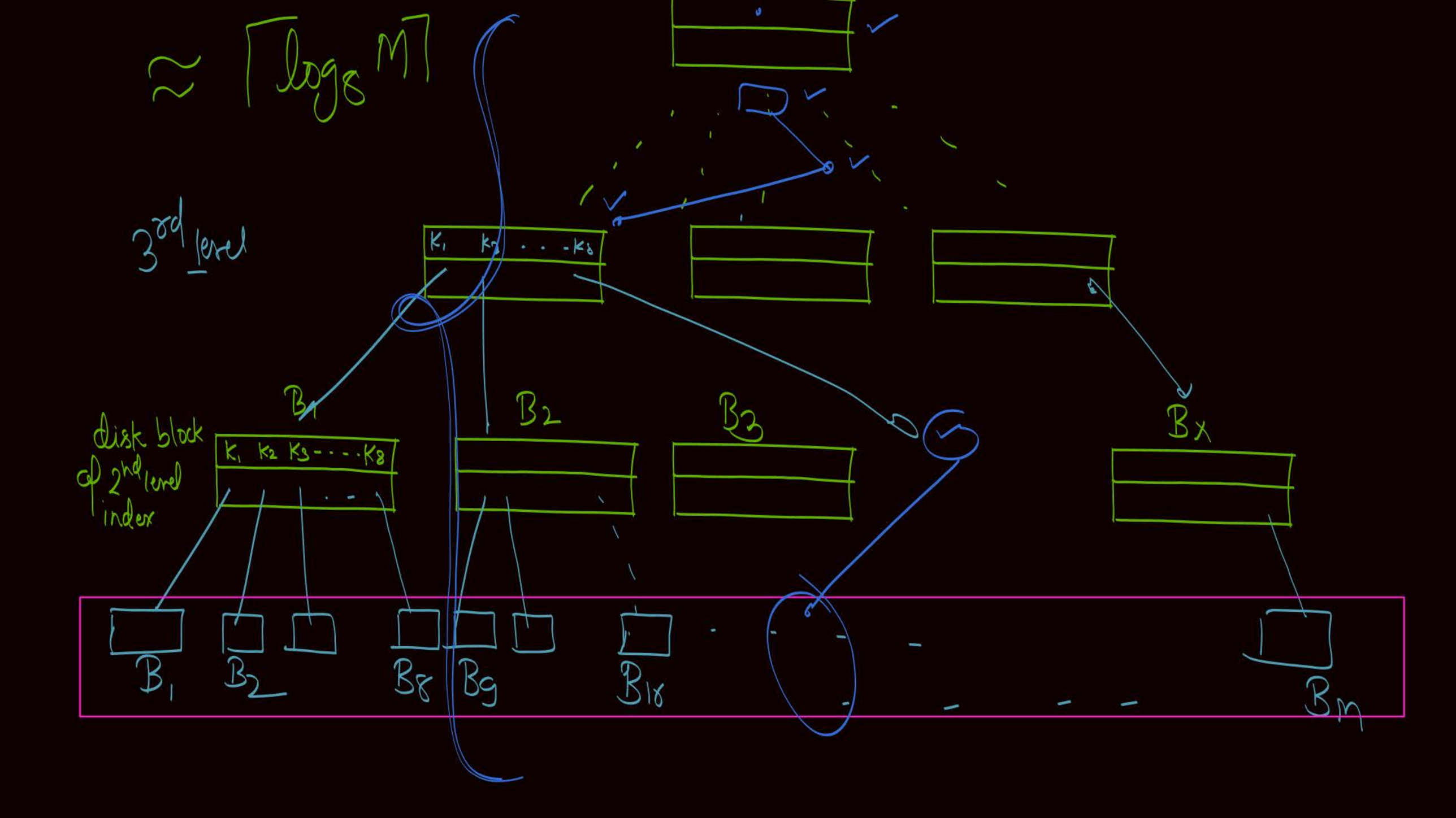
∴ Secondary index is always a dense index.

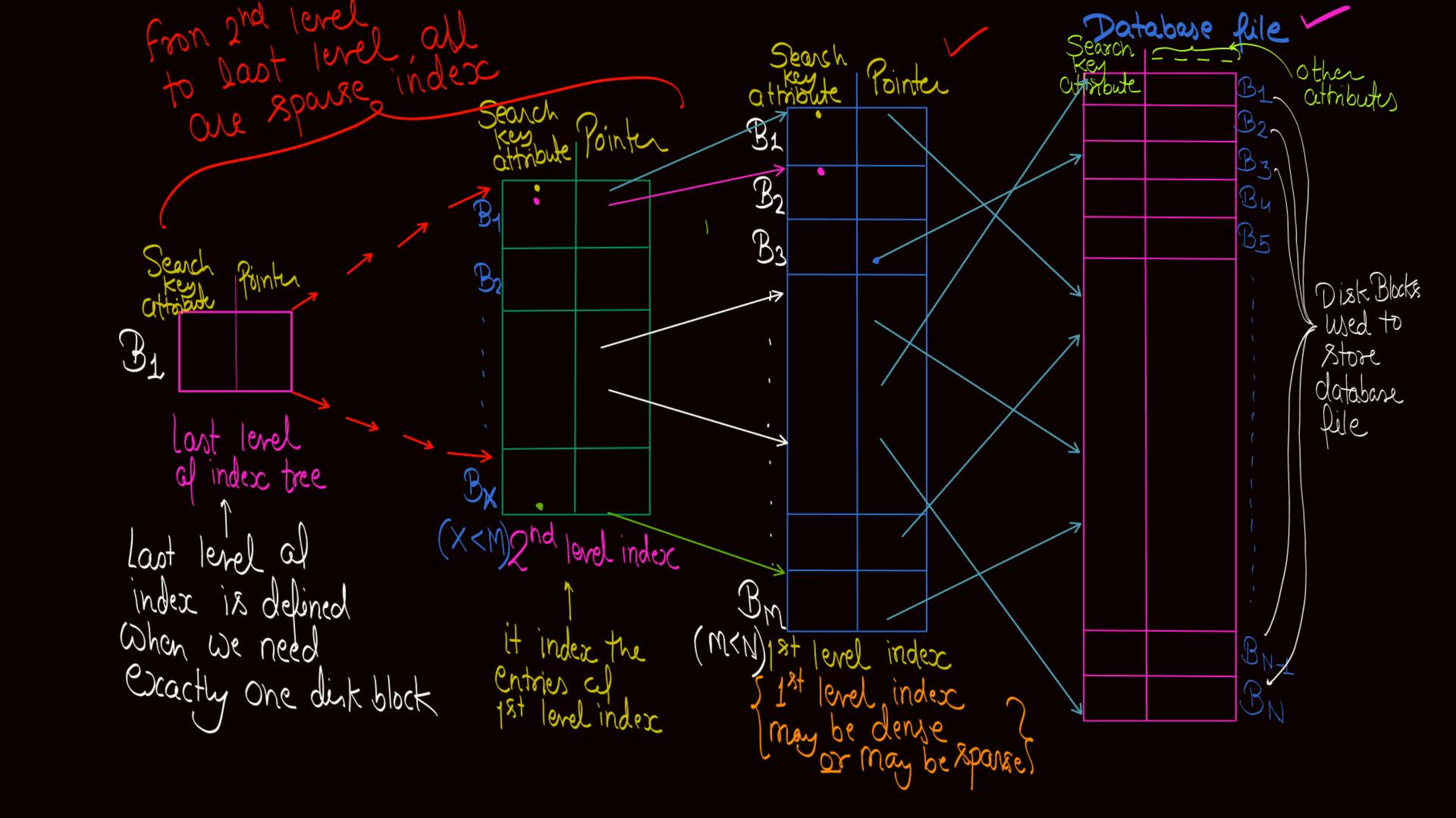
Note:- Any number of secondary index are possible on a database file {Because file don't need to be ordered}



Topic : Multi-level Index

Multi-level index is used to reduce the IO Cost w.r.t. single level index


≃ ⌈log8 M⌉
3rd level
K1 K2 . . . K8
B1
B2
B3
Bx
disk block of 2nd level index
K1 K2 K3 - - - - K8
B1
B2
B8
B9
B18
Bn


From 2nd level to last level, all are sparse index
Search key attribute
Pointer
B1
Last level of index tree
Last level of index is defined when we need exactly one disk block
Search key attribute
Pointer
B1
B2
Bx
(x<M) 2nd level index
it index the entries of 1st level index
Search key attribute
Pointer
B1
B2
B3
BM
(M<N) 1st level index
{ 1st level index may be dense or may be sparse }
Database file
Search key attribute
other attributes
B1
B2
B3
B4
B5
BN-1
BN
Disk Blocks used to store database file

Note :→

IO Cost using multi-level index

IO Cost using multi-level Index = Number of levels in multi-level index + 1

Number of levels in multi-level index: To locate an entry in the 1st level index of multi-level index

+ 1: For a block of database file to access corresponding record

Note :→

IO Cost using multi-level index

IO Cost using multi-level Index = Number of levels in multi-level index + 1

it is same for all access

i.e, same IO Cost for all three cases

Best Case, Worst Case, Average Case



Topic : Multi-level Index

In multi-level index,

- (1) 1st level of index may be dense or may be sparse
- (2) Except first level, all other levels of index are always sparse index



2 mins Summary

- Topic: Types of index
- Topic: Primary index
- Topic: Clustering index
- Topic: Secondary index
- Topic: Multi-level index

Slide

THANK - YOU